

IMPACT ANALYSIS

CR-032: Ascending and Descending Order Sorting of the Admin Student List

1. Change Summary

CR-032 introduces client-side sorting functionality to the administrator's student list.

Administrators will be able to sort the already-loaded student records in **Ascending** and **Descending** order based on supported fields such as **Registration Number** and **Student Name**.

The sorting will be performed on the existing student records without requiring changes to the backend API or database.

2. Impact Assessment

Area	Impact	Assessment
Frontend	Low – Moderate	Sorting controls and client-side sorting logic will be added to the Admin Dashboard.
Backend	None	Existing backend functionality remains unchanged.
API	None	Existing GET /api/students/ endpoint remains in use.
Database	None	No database schema or data changes are required.
Authentication	None	Existing administrator authentication remains unchanged.
Authorization	None	Existing administrator access protection remains unchanged.
Performance	Low	Sorting is performed locally on the already-loaded student records.
CRUD	Low	Edit/Delete must continue using the correct student_id after sorting.
Deployment	Low	No database migration or new backend endpoint is required.

3. Frontend Impact

The main affected area is the **Administrator Dashboard**.

Potential changes include:

- Ascending sorting control.
- Descending sorting control.
- Sorting by Registration Number.
- Sorting by Student Name.
- Client-side sorting logic.

- Sorting state management.
- Responsive styling for the sorting controls.

The implementation must preserve the relationship between each displayed student record and its original `student_id`.

Sorting must only change the display order of the records and must not modify the actual student data.

4. Backend, API and Database Impact

No backend changes are expected.

The existing:

`GET /api/students/`

endpoint will continue to provide the complete student list.

No new API endpoint or server-side sorting functionality is required.

No database schema or student data changes are required.

The sorting operation will be performed on the records that have already been loaded into the frontend.

5. Security Impact

The existing authentication and authorization mechanisms remain unchanged.

The student list must continue to be accessible only to authorized administrators.

The sorting feature must:

- Not bypass existing authorization.
- Not expose additional sensitive information.
- Preserve existing authentication mechanisms.
- Use normal React rendering.
- Preserve existing backend security controls.
- Not modify or expose the `student_id` unnecessarily.

6. CRUD and Functional Impact

Existing administrator CRUD operations must continue to work after sorting.

Particular attention is required for:

- Editing a student after sorting.
- Deleting a student after sorting.
- Maintaining the correct `student_id`.
- Ensuring sorting does not modify student data.

Sorting must only change the **display order** of student records.

It must not change the identity or stored information of any student.

7. Performance Impact

The expected performance impact is **low** because sorting occurs against student records that are already loaded in the browser.

No additional API request should be generated for each sorting action.

This approach is suitable for the current expected dataset size.

If the number of student records becomes significantly large in the future, server-side sorting or pagination may be considered as a separate change request.

8. Testing Impact

Manual testing should cover:

- Administrator login.
- Ascending sorting.
- Descending sorting.
- Registration Number Ascending order.
- Registration Number Descending order.
- Student Name Ascending order.
- Student Name Descending order.
- Correct ordering of all visible records.
- Sorting without additional API requests.
- Edit after sorting.
- Delete after sorting.
- Existing CRUD functionality.
- Administrator authentication and authorization.
- Responsive behaviour of the sorting controls.

9. Risk Analysis

Risk	Impact	Mitigation
------	--------	------------

Wrong student affected after sorting	High	Preserve and use the original student_id.
Incorrect Ascending/Descending order	Medium	Test sorting with multiple student records and different values.
Existing CRUD functionality breaks	High	Perform regression testing after implementing sorting.
Sorting becomes slow for large datasets	Medium	Keep current implementation client-side; consider server-side sorting for larger datasets in the future.
Sorting changes the original student data	High	Sort only the displayed record collection without modifying stored student information.
Responsive UI problems	Low	Test the sorting interface on supported screen sizes.

10. Overall Impact and Recommendation

Overall Impact: Low

CR-032 is primarily a **frontend usability enhancement** for the Administrator Dashboard. It does not require database, backend API, authentication, or authorization changes.

The change is technically feasible and recommended for implementation, subject to formal approval and subsequent testing.

Key implementation consideration: Existing Edit and Delete operations must continue to affect the correct student after the list is sorted by preserving and using the original student_id.